In [73]:
```
#FIFA-Predictions 2022-ysawate
```

In [22]:
```
import pandas as pd
import numpy as np
import random
```

In [23]:
```
df=pd.read_csv("C:/Users/sbawa/Downloads/results.csv")
df.head(10)
```

Out[23]:

	date	home_team	away_team	home_score	away_score	tournament	city	country
0	03-02-1900	Scotland	Wales	5	2	British Championship	Aberdeen	Scotland
1	24-02-1900	Wales	Northern Ireland	2	0	British Championship	Llandudno	Wales
2	03-03-1900	Northern Ireland	Scotland	0	3	British Championship	Belfast	Republic of Ireland
3	17-03-1900	Northern Ireland	England	0	2	British Championship	Dublin	Republic of Ireland
4	26-03-1900	Wales	England	1	1	British Championship	Cardiff	Wales
5	07-04-1900	Scotland	England	4	1	British Championship	Glasgow	Scotland
6	23-02-1901	Scotland	Northern Ireland	11	0	British Championship	Glasgow	Scotland
7	02-03-1901	Wales	Scotland	1	1	British Championship	Wrexham	Wales
8	09-03-1901	England	Northern Ireland	3	0	British Championship	Southampton	England
9	18-03-1901	England	Wales	6	0	British Championship	Newcastle	England

In [24]:
```
df.describe()
```

Out[24]:

	home_score	away_score
count	42301.000000	42301.000000
mean	1.740243	1.182549

	home_score	away_score
std	1.748487	1.399040
min	0.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	1.000000
75%	2.000000	2.000000
max	31.000000	21.000000

In [25]:

```
#Preprocessing
#Finding null values

df.isnull().sum()
```

Out[25]:

```
date          0
home_team     0
away_team     0
home_score    0
away_score    0
tournament    0
city          0
country       0
dtype: int64
```

In [26]:

```
#Let's remove those outlier,It will make our Machine Learning model more
accurate
#Asume maximum goals that one team can score is 15
higher_home=15
higher_away=15
df=df[(df["home_score"]<higher_home) & (df["away_score"]<higher_away)]
df.head(5)
#Dataset after removing outliers
```

Out[26]:

	date	home_team	away_team	home_score	away_score	tournament	city	country
0	03-02-1900	Scotland	Wales	5	2	British Championship	Aberdeen	Scotland
1	24-02-1900	Wales	Northern Ireland	2	0	British Championship	Llandudno	Wales
2	03-03-1900	Northern Ireland	Scotland	0	3	British Championship	Belfast	Republic of Ireland
3	17-03-1900	Northern Ireland	England	0	2	British Championship	Dublin	Republic of Ireland

	date	home_team	away_team	home_score	away_score	tournament	city	country
4	26-03-1900	Wales	England	1	1	British Championship	Cardiff	Wales

In [27]:

```
conditions  = [df["home_score"] ==df["away_score"], df["home_score"] >
df["away_score"] , df["home_score"] < df["away_score"]]
choices     = [ "Draw", 'Win', 'Lost' ]
df["Win_Status"] = np.select(conditions, choices)
df.head(5)
```

Out[27]:

	date	home_team	away_team	home_score	away_score	tournament	city	country	Win
0	03-02-1900	Scotland	Wales	5	2	British Championship	Aberdeen	Scotland	
1	24-02-1900	Wales	Northern Ireland	2	0	British Championship	Llandudno	Wales	
2	03-03-1900	Northern Ireland	Scotland	0	3	British Championship	Belfast	Republic of Ireland	
3	17-03-1900	Northern Ireland	England	0	2	British Championship	Dublin	Republic of Ireland	
4	26-03-1900	Wales	England	1	1	British Championship	Cardiff	Wales	

In [28]:

```
#See what are the Countries in the dataset

countries=df.home_team.unique()
print(f"There are {len(countries)} Countries in the home_team Column\n")
print(f"Countries-{countries}")
```

```
There are 306 Countries in the home_team Column

Countries-['Scotland' 'Wales' 'Northern Ireland' 'England' 'Uruguay' 'Austria'
 'Hungary' 'Argentina' 'Belgium' 'France' 'Netherlands' 'Czechoslovakia'
 'Switzerland' 'Sweden' 'Germany' 'Italy' 'Chile' 'Norway' 'Finland'
 'Luxembourg' 'Russia' 'Denmark' 'Catalonia' 'Basque Country' 'Brazil'
 'Paraguay' 'Japan' 'Canada' 'Estonia' 'Costa Rica' 'Guatemala' 'Spain'
 'Brittany' 'Poland' 'Yugoslavia' 'New Zealand' 'Romania' 'Latvia'
 'Galicia' 'Portugal' 'Andalusia' 'China PR' 'Australia' 'Lithuania'
 'Turkey' 'Central Spain' 'Mexico' 'Aruba' 'Egypt' 'Haiti' 'Philippines'
 'Bulgaria' 'United States' 'Jamaica' 'Kenya' 'Bolivia' 'Peru' 'Honduras'
 'Guyana' 'Uganda' 'Belarus' 'El Salvador' 'Barbados'
 'Republic of Ireland' 'Trinidad and Tobago' 'Greece' 'Curaçao' 'Dominica'
 'Silesia' 'Guadeloupe' 'Israel' 'Suriname' 'French Guiana' 'Cuba'
```

```
'Colombia' 'Ecuador' 'Saint Kitts and Nevis' 'Panama' 'Slovakia'
'Manchukuo' 'Croatia' 'Nicaragua' 'Afghanistan' 'India' 'Martinique'
'Zimbabwe' 'Iceland' 'Albania' 'Madagascar' 'Zambia' 'Mauritius'
'Tanzania' 'Iran' 'Djibouti' 'DR Congo' 'Vietnam' 'Macau' 'Ethiopia'
'Puerto Rico' 'Réunion' 'Sierra Leone' 'Zanzibar' 'South Korea' 'Ghana'
'South Africa' 'New Caledonia' 'Fiji' 'Nigeria' 'Venezuela' 'Burma'
'Sri Lanka' 'Tahiti' 'Gambia' 'Hong Kong' 'Singapore' 'Malaysia'
'Indonesia' 'Guinea-Bissau' 'German DR' 'Vanuatu' 'Kernow' 'Saarland'
'Cambodia' 'Lebanon' 'Pakistan' 'Vietnam Republic' 'North Korea' 'Togo'
'Sudan' 'Malta' 'Syria' 'Tunisia' 'Malawi' 'Morocco' 'Benin' 'Cameroon'
'Central African Republic' 'Gabon' 'Ivory Coast' 'Congo' 'Mali'
'North Vietnam' 'Mongolia' 'Taiwan' 'Cyprus' 'Iraq' 'Saint Lucia'
'Grenada' 'Thailand' 'Senegal' 'Libya' 'Guinea' 'Algeria' 'Kuwait'
'Jordan' 'Solomon Islands' 'Liberia' 'Laos'
'Saint Vincent and the Grenadines' 'Bermuda' 'Niger' 'Bahrain'
'Montenegro' 'Palestine' 'Papua New Guinea' 'Burkina Faso' 'Mauritania'
'Saudi Arabia' 'Eswatini' 'Somalia' 'Lesotho' 'Qatar'
'Antigua and Barbuda' 'Faroe Islands' 'Bangladesh' 'Oman' 'Yemen DPR'
'Burundi' 'Yemen' 'Mozambique' 'Guam' 'Chad' 'Angola'
'Dominican Republic' 'Seychelles' 'Rwanda' 'São Tomé and Príncipe'
'Botswana' 'Northern Cyprus' 'Cape Verde' 'Kyrgyzstan' 'Georgia'
'Azerbaijan' 'Kiribati' 'Tonga' 'Wallis Islands and Futuna'
'United Arab Emirates' 'Brunei' 'Equatorial Guinea' 'Liechtenstein'
'Nepal' 'Greenland' 'Niue' 'Samoa' 'American Samoa' 'Belize' 'Anguilla'
'Cayman Islands' 'Palau' 'Sint Maarten' 'Namibia' 'Åland Islands'
'Ynys Môn' 'Saint Martin' 'San Marino' 'Slovenia' 'Jersey' 'Shetland'
'Isle of Wight' 'Moldova' 'Ukraine' 'Kazakhstan' 'Tajikistan'
'Uzbekistan' 'Turkmenistan' 'Armenia' 'Czech Republic' 'Guernsey'
'Gibraltar' 'Isle of Man' 'North Macedonia' 'Montserrat' 'Serbia'
'Cook Islands' 'Canary Islands' 'Bosnia and Herzegovina' 'Maldives'
'Andorra' 'British Virgin Islands' 'Frøya' 'Hitra'
'United States Virgin Islands' 'Corsica' 'Eritrea' 'Bahamas' 'Gotland'
'Saare County' 'Rhodes' 'Micronesia' 'Bhutan' 'Orkney' 'Monaco' 'Tuvalu'
'Alderney' 'Mayotte' 'Turks and Caicos Islands' 'East Timor'
'Western Isles' 'Falkland Islands' 'Kosovo' 'Republic of St. Pauli'
'Găgăuzia' 'Tibet' 'Crimea' 'Occitania' 'Sápmi'
'Northern Mariana Islands' 'Menorca' 'Comoros' 'Provence'
'Arameans Suryoye' 'Padania' 'Iraqi Kurdistan' 'Gozo' 'Bonaire'
'Western Sahara' 'Raetia' 'Tamil Eelam' 'South Sudan' 'Abkhazia'
'Artsakh' 'Madrid' 'Vatican City' 'Ellan Vannin' 'South Ossetia' 'Darfur'
'County of Nice' 'Székely Land' 'Romani people' 'Felvidék'
'Chagos Islands' 'United Koreans in Japan' 'Somaliland' 'Western Armenia'
'Barawa' 'Kárpátalja' 'Yorkshire' 'Panjab' 'Matabeleland' 'Cascadia'
'Kabylia' 'Timor-Leste' 'Myanmar' 'Parishes of Jersey' 'Chameria'
'Saint Helena']
```

In [29]:

```
years = []
for date in df.date:
    years.append(int(str(date).split('-')[-1]))
```

In [30]:

```
#Now take the rpws that home_team==Country
#Nad romove the data of friendly matches
```

```
data_home=df.loc[df["home_team"]==df["country"] ]
data_home=df.loc[df["tournament"] != "Friendly"]
data_home.head(5)
```

Out[30]:

	date	home_team	away_team	home_score	away_score	tournament	city	country	Win
0	03-02-1900	Scotland	Wales	5	2	British Championship	Aberdeen	Scotland	
1	24-02-1900	Wales	Northern Ireland	2	0	British Championship	Llandudno	Wales	
2	03-03-1900	Northern Ireland	Scotland	0	3	British Championship	Belfast	Republic of Ireland	
3	17-03-1900	Northern Ireland	England	0	2	British Championship	Dublin	Republic of Ireland	
4	26-03-1900	Wales	England	1	1	British Championship	Cardiff	Wales	

In [31]:

```
#How home team performs in tournament matches

tournament_df=pd.crosstab(data_home["tournament"],
data_home["Win_Status"],margins=True)
tournament_df=tournament_df.sort_values("All",ascending=False).head(10)
tournament_df.style.bar(color="brown",subset=
["Draw","Lost","Win","All"])
```

Out[31]:

Win_Status tournament	Draw	Lost	Win	All
All	5429	7152	12423	25004
FIFA World Cup qualification	1590	2037	3844	7471
UEFA Euro qualification	506	815	1261	2582
African Cup of Nations qualification	418	353	948	1719
FIFA World Cup	199	290	411	900
Copa América	181	231	429	841

Win_Status tournament	Draw	Lost	Win	All
AFC Asian Cup qualification	123	174	424	721
African Cup of Nations	193	186	311	690
CECAFA Cup	147	185	288	620
CFU Caribbean Cup qualification	124	170	310	604

In [32]:

```
data_home['Win_Status'].value_counts()
```

Out[32]:

```
Win     12423
Lost     7152
Draw     5429
Name: Win_Status, dtype: int64
```

In [33]:

```
#How each team perform while playing a tournament in home grounds

teams_win_status=pd.crosstab(df["home_team"],
df["Win_Status"],margins=True, margins_name="Total")
teams_win_status["team_win_probability"]=teams_win_status["Win"]/(teams_wi

#Lets take teams which plays atleast 200 games
teams_win_status_100=teams_win_status.loc[teams_win_status["Total"]>200]
teams_win_status_100=teams_win_status_100.sort_values("team_win_probabilit

teams_win_status_100.head(20).style.bar(color="orange",subset="team_win_pr



#We can see teams has more than 50% wining probability while playing in
the home grounds
```

Out[33]:

Win_Status home_team	Draw	Lost	Win	Total	team_win_probability
Brazil	110	57	414	581	0.712565
Spain	69	49	251	369	0.680217
Argentina	123	68	366	557	0.657092
Ivory Coast	64	35	176	275	0.640000
Iran	61	42	180	283	0.636042
Italy	120	49	287	456	0.629386

Win_Status home_team	Draw	Lost	Win	Total	team_win_probability
Nigeria	67	34	171	272	0.628676
Egypt	68	74	239	381	0.627297
Germany	111	85	322	518	0.621622
England	107	75	296	478	0.619247
Ghana	72	39	177	288	0.614583
Russia	70	50	187	307	0.609121
Czechoslovakia	51	40	139	230	0.604348
Morocco	68	47	173	288	0.600694
South Korea	112	79	279	470	0.593617
Senegal	55	36	132	223	0.591928
Mexico	116	98	306	520	0.588462
Costa Rica	77	50	179	306	0.584967
Sweden	105	102	290	497	0.583501
Iraq	72	44	162	278	0.582734

In [34]:

```
#Teams playing in away conditions

teams_away_status=pd.crosstab(df["away_team"],
df["Win_Status"],margins=True, margins_name="Total")
teams_away_status["team_win_probability"]=teams_away_status["Lost"]/(teams

#Lets take teams which plays atleast 200 games
teams_away_status_100=teams_away_status.loc[teams_away_status["Total"]>200

teams_away_status_100=teams_away_status_100.sort_values("team_win_probabil

teams_away_status_100.rename(columns={'Lost': 'Win'}, index={'Win':
'Lost'}, inplace=True)
teams_away_status_100.head(20)

#The probability to wining a tournament in away conditions is very low
comapred to the winning probalities in home conditions
```

Out[34]:

Win_Status away_team	Draw	Win	Win	Total	team_win_probability
Germany	89	241	119	449	0.536748
Brazil	92	224	102	418	0.535885

Win_Status away_team	Draw	Win	Win	Total	team_win_probability
England	129	248	107	484	0.512397
Spain	99	163	82	344	0.473837
South Korea	110	178	109	397	0.448363
Iran	67	97	57	221	0.438914
Netherlands	81	163	129	373	0.436997
Russia	116	175	112	403	0.434243
Japan	53	111	93	257	0.431907
Italy	109	151	104	364	0.414835
Australia	56	93	76	225	0.413333
Sweden	121	220	198	539	0.408163
Yugoslavia	61	115	115	291	0.395189
France	82	143	139	364	0.392857
Zambia	105	166	155	426	0.389671
Mexico	87	141	134	362	0.389503
Portugal	64	115	119	298	0.385906
Argentina	126	168	145	439	0.382688
Hungary	106	183	195	484	0.378099
Ivory Coast	81	108	107	296	0.364865

In [35]:

```
#Making a plot to identify wining possibilities in both home and away
conditions

win_prob={teams_win_status_100.iloc[i,:].name:
[teams_win_status_100.iloc[i,4]] for i in
range(teams_win_status_100.shape[0])}
for i in range(teams_away_status_100.shape[0]):
    try:

win_prob[teams_away_status_100.iloc[i,:].name].append(teams_away_status_10

    except:
        pass
country_names=win_prob.keys()
lst_home=[i[0] for i in win_prob.values()]

lst_away=[]
for i in win_prob.values():
```

```
    if len(i)==2:
        lst_away.append(i[1])
    else:
        lst_away.append(0)
```

In [36]:

```
range_years=max(years)-min(years)
no_0f_terms=10
term_size=int(range_years/no_0f_terms)
for i in range(no_0f_terms+1):
    start=years.index(term_size*i+min(years))
    end=years.index(min(term_size*(i+1)+min(years),2021))
    term=df.iloc[start:end]
    best_teams=pd.crosstab(term["home_team"],
term["Win_Status"],margins=True, margins_name="Total")

    ##Lets take teams which plays atleast 20 games

best_teams["team_win_probability"]=best_teams["Win"]/(best_teams["Total"])



best_teams=best_teams.sort_values("team_win_probability",ascending=False)

    best_teams=best_teams.loc[best_teams["Total"]>20]
    if (best_teams.shape[0]>2):
        print(f"\nBest 2 team in the term: {term_size*i+min(years)}-
{min(term_size*(i+1)+min(years),2021)} ")
        print(best_teams.iloc[0:2].to_markdown())
    else:
        print(f"No Enough data to find the best team in the term:
{term_size*i+min(years)}-{min(term_size*(i+1)+min(years),2021)} ")
```

```
No Enough data to find the best team in the term: 1900-1912

Best 2 team in the term: 1912-1924
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Hungary      |      4 |      4 |    19 |      27 |               0.703704 |
| Uruguay      |      4 |      7 |    24 |      35 |               0.685714 |

Best 2 team in the term: 1924-1936
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Sweden       |      5 |      7 |    38 |      50 |               0.76     |
| Argentina    |      7 |      4 |    28 |      39 |               0.717949 |
```

```
Best 2 team in the term: 1936-1948
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Argentina    |      3 |      2 |    41 |      46 |               0.891304 |
| Italy        |      4 |      0 |    17 |      21 |               0.809524 |

Best 2 team in the term: 1948-1960
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Costa Rica   |      1 |      1 |    19 |      21 |               0.904762 |
| Hungary      |      6 |      4 |    37 |      47 |               0.787234 |

Best 2 team in the term: 1960-1972
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Guinea       |      2 |      3 |    18 |      23 |               0.782609 |
| Brazil       |     12 |      7 |    57 |      76 |               0.75     |

Best 2 team in the term: 1972-1984
| home_team      |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:--------------|-------:|-------:|------:|--------:|----------------------:|
| South Korea    |     18 |      6 |    73 |      97 |               0.752577 |
| New Caledonia |      0 |      7 |    18 |      25 |               0.72     |

Best 2 team in the term: 1984-1996
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Nigeria      |      9 |      2 |    34 |      45 |               0.755556 |
| Romania      |      9 |      5 |    36 |      50 |               0.72     |

Best 2 team in the term: 1996-2008
| home_team       |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:---------------|-------:|-------:|------:|--------:|----------------------:|
| Brazil          |     10 |      9 |    83 |     102 |               0.813725 |
| Czech Republic |      6 |     12 |    55 |      73 |               0.753425 |

Best 2 team in the term: 2008-2020
| home_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Spain        |      9 |      7 |    65 |      81 |               0.802469 |
| Egypt        |     12 |     10 |    75 |      97 |               0.773196 |
No Enough data to find the best team in the term: 2020-2021
```

In [37]:

```
range_years=max(years)-min(years)
no_0f_terms=10
term_size=int(range_years/no_0f_terms)
for i in range(no_0f_terms+1):
    start=years.index(term_size*i+min(years))
    end=years.index(min(term_size*(i+1)+min(years),2021))
    term=df.iloc[start:end]
    best_teams=pd.crosstab(term["away_team"],
term["Win_Status"],margins=True, margins_name="Total")
```

```
    ##Lets take teams which plays atleast 200 games

best_teams["team_win_probability"]=best_teams["Lost"]/(best_teams["Total"]


best_teams=best_teams.sort_values("team_win_probability",ascending=False)

    best_teams=best_teams.loc[best_teams["Total"]>20]
    print(f"\nBest 2 team in the term: {term_size*i+min(years)}-
{min(term_size*(i+1)+min(years),2021)} ")
    if (best_teams.shape[0]>2):
        print(best_teams.iloc[0:2].to_markdown())
    else:
        print(f"No Enough data to find the best team in the term:
{term_size*i+min(years)}-{min(term_size*(i+1)+min(years),2021)} ")
```

```
Best 2 team in the term: 1900-1912
No Enough data to find the best team in the term: 1900-1912

Best 2 team in the term: 1912-1924
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Hungary     |     10 |     13 |     7 |      30 |               0.433333 |
| Uruguay     |      9 |     12 |    16 |      37 |               0.324324 |

Best 2 team in the term: 1924-1936
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| England     |      6 |     22 |    10 |      38 |               0.578947 |
| Scotland    |      5 |     16 |     8 |      29 |               0.551724 |

Best 2 team in the term: 1936-1948
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Germany     |      2 |     22 |    11 |      35 |               0.628571 |
| England     |      1 |     14 |    11 |      26 |               0.538462 |

Best 2 team in the term: 1948-1960
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Hungary     |     12 |     34 |    12 |      58 |               0.586207 |
| South Korea |      7 |     13 |     5 |      25 |               0.52     |

Best 2 team in the term: 1960-1972
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| England     |     12 |     38 |    12 |      62 |               0.612903 |
| Brazil      |      9 |     34 |    15 |      58 |               0.586207 |

Best 2 team in the term: 1972-1984
```

```
| away_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Tahiti       |      3 |     21 |     5 |      29 |               0.724138 |
| Germany      |     15 |     32 |    11 |      58 |               0.551724 |

Best 2 team in the term: 1984-1996
| away_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Brazil       |     13 |     32 |    15 |      60 |               0.533333 |
| Netherlands |     10 |     26 |    15 |      51 |               0.509804 |

Best 2 team in the term: 1996-2008
| away_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| France       |     15 |     40 |     8 |      63 |               0.634921 |
| Netherlands |     16 |     30 |    12 |      58 |               0.517241 |

Best 2 team in the term: 2008-2020
| away_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Spain        |     13 |     58 |    12 |      83 |               0.698795 |
| Brazil       |     10 |     50 |    12 |      72 |               0.694444 |

Best 2 team in the term: 2020-2021
No Enough data to find the best team in the term: 2020-2021
```

In [38]:

```
range_years=max(years)-min(years)
no_0f_terms=10
term_size=int(range_years/no_0f_terms)
for i in range(no_0f_terms+1):
    start=years.index(term_size*i+min(years))
    end=years.index(min(term_size*(i+1)+min(years),2021))
    term=df.iloc[start:end]
    best_teams=pd.crosstab(term["away_team"],
term["Win_Status"],margins=True, margins_name="Total")

    ##Lets take teams which plays atleast 200 games

best_teams["team_win_probability"]=best_teams["Lost"]/(best_teams["Total"]



best_teams=best_teams.sort_values("team_win_probability",ascending=False)

    best_teams=best_teams.loc[best_teams["Total"]>20]
    print(f"\nBest 2 team in the term: {term_size*i+min(years)}-
{min(term_size*(i+1)+min(years),2021)} ")
    if (best_teams.shape[0]>2):
        print(best_teams.iloc[0:2].to_markdown())
```

```
    else:
        print(f"No Enough data to find the best team in the term: {term_size*i+min(years)}-{min(term_size*(i+1)+min(years),2021)} ")
```

```
Best 2 team in the term: 1900-1912
No Enough data to find the best team in the term: 1900-1912

Best 2 team in the term: 1912-1924
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Hungary     |     10 |     13 |     7 |      30 |               0.433333 |
| Uruguay     |      9 |     12 |    16 |      37 |               0.324324 |

Best 2 team in the term: 1924-1936
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| England     |      6 |     22 |    10 |      38 |               0.578947 |
| Scotland    |      5 |     16 |     8 |      29 |               0.551724 |

Best 2 team in the term: 1936-1948
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Germany     |      2 |     22 |    11 |      35 |               0.628571 |
| England     |      1 |     14 |    11 |      26 |               0.538462 |

Best 2 team in the term: 1948-1960
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Hungary     |     12 |     34 |    12 |      58 |               0.586207 |
| South Korea |      7 |     13 |     5 |      25 |               0.52     |

Best 2 team in the term: 1960-1972
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| England     |     12 |     38 |    12 |      62 |               0.612903 |
| Brazil      |      9 |     34 |    15 |      58 |               0.586207 |

Best 2 team in the term: 1972-1984
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Tahiti      |      3 |     21 |     5 |      29 |               0.724138 |
| Germany     |     15 |     32 |    11 |      58 |               0.551724 |

Best 2 team in the term: 1984-1996
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| Brazil      |     13 |     32 |    15 |      60 |               0.533333 |
| Netherlands |     10 |     26 |    15 |      51 |               0.509804 |

Best 2 team in the term: 1996-2008
| away_team   |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|-----------------------:|
| France      |     15 |     40 |     8 |      63 |               0.634921 |
| Netherlands |     16 |     30 |    12 |      58 |               0.517241 |
```

```
Best 2 team in the term: 2008-2020
| away_team    |   Draw |   Lost |   Win |   Total |   team_win_probability |
|:------------|-------:|-------:|------:|--------:|----------------------:|
| Spain        |     13 |     58 |    12 |      83 |               0.698795 |
| Brazil       |     10 |     50 |    12 |      72 |               0.694444 |

Best 2 team in the term: 2020-2021
No Enough data to find the best team in the term: 2020-2021
```

In [39]:

```
#ML MODEL
```

In [40]:

```
df_match=df.copy() #making a copy of the main dataframe for future use
df_match.head(8)
```

Out[40]:

	date	home_team	away_team	home_score	away_score	tournament	city	country	Win
0	03-02-1900	Scotland	Wales	5	2	British Championship	Aberdeen	Scotland	
1	24-02-1900	Wales	Northern Ireland	2	0	British Championship	Llandudno	Wales	
2	03-03-1900	Northern Ireland	Scotland	0	3	British Championship	Belfast	Republic of Ireland	
3	17-03-1900	Northern Ireland	England	0	2	British Championship	Dublin	Republic of Ireland	
4	26-03-1900	Wales	England	1	1	British Championship	Cardiff	Wales	
5	07-04-1900	Scotland	England	4	1	British Championship	Glasgow	Scotland	
6	23-02-1901	Scotland	Northern Ireland	11	0	British Championship	Glasgow	Scotland	
7	02-03-1901	Wales	Scotland	1	1	British Championship	Wrexham	Wales	

In [41]:

```
New_Dataset_part_1=pd.DataFrame(list(zip(years,df_match.values[:,7],df_mat
["year","Country","team_1","team_2","team_1_score","team_2_score"])
#Making a new dataset by changing the team_1 and team_2 and their
respective scores
New_Dataset_part_2=pd.DataFrame(list(zip(years,df_match.values[:,7],df_mat
["year","Country","team_1","team_2","team_1_score","team_2_score"])
New_Dataset=pd.concat([New_Dataset_part_1,New_Dataset_part_2],axis=0)
New_Dataset =New_Dataset.sample(frac=1).reset_index(drop=True)
```

```
#Shaffling the dataset
New_Dataset.head(5)
```

Out[41]:

	year	Country	team_1	team_2	team_1_score	team_2_score
0	1975	Mexico	Costa Rica	United States	3	1
1	2011	Maldives	Maldives	Seychelles	3	0
2	1978	Republic of Ireland	Turkey	Republic of Ireland	2	4
3	1984	Ivory Coast	Senegal	Ivory Coast	0	1
4	1998	Slovenia	Slovenia	Latvia	1	0

In [42]:

```
teams_1=New_Dataset.team_1.unique()
contries=New_Dataset.Country.unique()
all_countries=np.unique(np.concatenate((teams_1,contries), axis=0))
len(all_countries)
```

Out[42]: 343

In [43]:

```
#Defining the features and labels(Targets)

Y= New_Dataset.iloc[:,4:6] #Training targets (team_1_score and
team_2_score)
categorized_data=New_Dataset.iloc[:,0:4].copy() #Traing features

from sklearn import preprocessing
label_encoder = preprocessing.LabelEncoder()

#Labeling the data using LabelEncorder in Sklearn-(Giving a unique
number to each string(country))

label_encoder.fit(all_countries)
#list(label_encoder.classes_)
categorized_data['team_1']=label_encoder.transform(categorized_data['team_

categorized_data['team_2']=label_encoder.transform(categorized_data['team_

categorized_data['Country']=label_encoder.transform(categorized_data['Coun



#Converting these feature columns to categrize form to make the training
processs more smoother
categorized_data['team_1']=categorized_data['team_1'].astype("category")
```

```
categorized_data['team_2']=categorized_data['team_2'].astype("category")
categorized_data['Country']=categorized_data['team_2'].astype("category")
```

In [44]:

```
#Input Fatures to the model (x)

categorized_data.head(5)
```

Out[44]:

	year	Country	team_1	team_2
0	1975	313	69	313
1	2011	262	174	262
2	1978	238	303	238
3	1984	144	259	144
4	1998	159	269	159

In [45]:

```
#Targets to the model (Y)

Y.head(5)
```

Out[45]:

	team_1_score	team_2_score
0	3	1
1	3	0
2	2	4
3	0	1
4	1	0

In [46]:

```
#Info about the X and Y dataframes

print(categorized_data.info())
print(Y.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 84502 entries, 0 to 84501
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   year     84502 non-null  int64
 1   Country  84502 non-null  category
 2   team_1   84502 non-null  category
 3   team_2   84502 non-null  category
dtypes: category(3), int64(1)
memory usage: 1.2 MB
None
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 84502 entries, 0 to 84501
Data columns (total 2 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   team_1_score  84502 non-null  int64
 1   team_2_score  84502 non-null  int64
dtypes: int64(2)
memory usage: 1.3 MB
None
```

In [47]:

```
#Making the model

X=categorized_data
from sklearn.multioutput import MultiOutputRegressor
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier



model = MultiOutputRegressor(RandomForestClassifier())
model.fit(X,Y)
```

Out[47]:

```
MultiOutputRegressor(estimator=RandomForestClassifier())
```

In [48]:

```
#Making the predictions

prd=model.predict(X)
prd
```

Out[48]:

```
array([[3, 1],
       [3, 0],
       [2, 4],
       ...,
       [2, 1],
       [1, 0],
       [2, 2]], dtype=int64)
```

In [49]:

```
#Creating the Confusion matrix for each predictions

score_team_1=[i[0] for i in prd]
score_team_2=[i[1] for i in prd]

from sklearn.metrics import confusion_matrix
cm1=confusion_matrix(list(Y.iloc[:,0]),score_team_1)
cm2=confusion_matrix(list(Y.iloc[:,1]),score_team_2)
```

In [50]:
```
#Classification Report to team 1 Score

from sklearn.metrics import classification_report
report_1=classification_report(Y.iloc[:,0],score_team_1)
print(report_1)

#Has a good Accuracy in predicting the team 1 Score
```

```
              precision    recall  f1-score   support

           0       0.87      0.89      0.88     25925
           1       0.86      0.88      0.87     26170
           2       0.86      0.84      0.85     16628
           3       0.86      0.81      0.83      8121
           4       0.86      0.80      0.83      3857
           5       0.84      0.80      0.82      1760
           6       0.87      0.80      0.83       940
           7       0.88      0.81      0.85       494
           8       0.85      0.79      0.82       263
           9       0.86      0.81      0.83       150
          10       0.89      0.86      0.87        85
          11       0.88      0.93      0.90        40
          12       0.88      0.86      0.87        35
          13       0.90      0.90      0.90        20
          14       0.93      1.00      0.97        14

    accuracy                           0.86     84502
   macro avg       0.87      0.85      0.86     84502
weighted avg       0.86      0.86      0.86     84502
```

In [51]:
```
#Classification Report to team 2 Score

report_2=classification_report(Y.iloc[:,1],score_team_2)
print(report_2)

#Has a good Accuracy in predicting the team 1 Score#
```

```
              precision    recall  f1-score   support

           0       0.87      0.89      0.88     25925
           1       0.86      0.89      0.87     26170
           2       0.86      0.84      0.85     16628
           3       0.86      0.81      0.83      8121
           4       0.86      0.80      0.83      3857
           5       0.85      0.79      0.82      1760
           6       0.87      0.79      0.83       940
           7       0.87      0.82      0.85       494
           8       0.84      0.81      0.82       263
           9       0.84      0.84      0.84       150
          10       0.87      0.88      0.88        85
          11       0.88      0.93      0.90        40
```

```
           12       0.93      0.80      0.86        35
           13       0.90      0.90      0.90        20
           14       0.93      1.00      0.97        14

     accuracy                           0.86     84502
    macro avg       0.87      0.85      0.86     84502
 weighted avg       0.86      0.86      0.86     84502
```

In [64]:

```
#Fuction to Select the winning team for the prediction array

def select_winning_team(probability_array):
    prob_lst=[round(probability_array[0][i],3) for i in range(2)]
    if (prob_lst[0]>prob_lst[1]):
        out=0
    elif (prob_lst[0]<prob_lst[1]):
        out=1
    elif (prob_lst[0]==prob_lst[1]):
        out=2
    return out,prob_lst
```

In [82]:

```
#Sample Prediction

mactch_played=2015
team_1="England"
team_2="Spain"
stadium="Qatar"

team_lst=[team_1,team_2]
team_1_num=label_encoder.transform([team_1])[0]
team_2_num=label_encoder.transform([team_2])[0]
stadium_num=label_encoder.transform([stadium])[0]

print(f"Team 01 is {team_1} -{team_1_num}")
print(f"Team 02 is {team_2} -{team_2_num}")
print(f"Played in  {stadium} -{stadium_num}")
```

```
Team 01 is England -90
Team 02 is Spain -279
Played in  Qatar -236
```

In [83]:

```
#Sample Prediction Output

X_feature=np.array([[mactch_played,stadium_num,team_1_num,team_2_num]])
res=model.predict(X_feature)
```

```
win,_=select_winning_team(res)
try:
    print(f"{team_1} vs {team_2} \n {team_lst[win]} wins 🏆⚽🎯\n")
except IndexError:
    print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽⚽\n")
```

```
England vs Spain
  Match Draw ⚽⚽⚽
```

In [69]:

```
Group_A= ["Uruguay","Russia","Saudi Arabia","Egypt"]
Group_B= ["Portugal","Spain","Morocco","Iran"]
Group_C= ["France","Denmark","Australia","Peru"]
Group_D= ["Argentina","Croatia","Iceland","Nigeria"]
Group_E= ["Brazil","Switzerland","Costa Rica","Serbia"]
Group_F= ["South Korea","Mexico","Sweden","Germany"]
Group_G= ["Belgium","England","Panama","Tunisia"]
Group_H= ["Senegal","Japan","Poland","Colombia"]
Groups={"Group A":Group_A,"Group B":Group_B,"Group C":Group_C,"Group
D":Group_D,"Group E":Group_E,"Group F":Group_F,"Group G":Group_G,"Group
H":Group_H}
```

In [71]:

```
year=2022
stadium="Qatar"
stadium_num=label_encoder.transform([stadium])[0]
host_num=stadium_num
```

In [57]:

```
##Group stage Matches


Group_standings={}
for grp_name in list(Groups.keys()):
    print(f"{grp_name} Matches")
    probable_countries=Groups[grp_name]
    team_wins_dct={}
    goal_scored_dct={}
    goal_against_dct={}
    win_dct={}
    draw_dct={}
    lost_dct={}
    for i in range(len(probable_countries)):
        j=i+1
        team_1=probable_countries[i]
        team_1_num=label_encoder.transform([team_1])[0]
```

```
        team_wins=0
        while j<len((probable_countries)):
            team_2=probable_countries[j]
            team_2_num=label_encoder.transform([team_2])[0]
            team_lst=[team_1,team_2]

Input_vector=np.array([[year,stadium_num,team_1_num,team_2_num]])
            res=model.predict(Input_vector)

            win,prob_lst=select_winning_team(res)
            goal_scored_dct[team_1] =
goal_scored_dct.get(team_1,0)+prob_lst[0]
            goal_scored_dct[team_2] =
goal_scored_dct.get(team_2,0)+prob_lst[1]

            goal_against_dct[team_1] =
goal_against_dct.get(team_1,0)+prob_lst[1]
            goal_against_dct[team_2] =
goal_against_dct.get(team_2,0)+prob_lst[0]

            try:
                print(f" {team_1} vs {team_2} \n  Results of the Match
{res[0]}\n   {team_lst[win]} wins 🏟🏆⚽🎖\n")
                if (win)==0:
                    team_wins_dct[team_1] =
team_wins_dct.get(team_1,0)+2
                    team_wins_dct[team_2] = team_wins_dct.get(team_2,0)

                    win_dct[team_1] = win_dct.get(team_1,0)+1
                    win_dct[team_2] = win_dct.get(team_2,0)
                    lost_dct[team_2] = lost_dct.get(team_2,0)+1
                    lost_dct[team_1] = lost_dct.get(team_1,0)
                    draw_dct[team_2] = draw_dct.get(team_2,0)
                    draw_dct[team_1] = draw_dct.get(team_1,0)

                elif (win)==1:
                    team_wins_dct[team_2] =
team_wins_dct.get(team_2,0)+2
                    team_wins_dct[team_1] = team_wins_dct.get(team_1,0)

                    win_dct[team_2] = win_dct.get(team_2,0)+1
```

```
                    win_dct[team_1] = win_dct.get(team_1,0)
                    lost_dct[team_1] = lost_dct.get(team_1,0)+1
                    lost_dct[team_2] = lost_dct.get(team_2,0)
                    draw_dct[team_1] = draw_dct.get(team_1,0)
                    draw_dct[team_2] = draw_dct.get(team_2,0)

            except IndexError:
                print(f"{team_1} vs {team_2} \n  Results of the Match
{res[0]}\n   Match Draw ⚽⚽⚽\n")
                team_wins_dct[team_1] = team_wins_dct.get(team_1,0)+1
                team_wins_dct[team_2] = team_wins_dct.get(team_2,0)+1

                draw_dct[team_1] = draw_dct.get(team_1,0)+1
                draw_dct[team_2] = draw_dct.get(team_2,0)+1

                win_dct[team_1] = win_dct.get(team_1,0)
                lost_dct[team_1] = lost_dct.get(team_1,0)

                win_dct[team_2] = win_dct.get(team_2,0)
                lost_dct[team_2] = lost_dct.get(team_2,0)

            j=j+1
    group_results=
[win_dct,draw_dct,lost_dct,team_wins_dct,goal_scored_dct,goal_against_dct]

    Group_standings[grp_name]=group_results
```

```
Group A Matches
 England vs Russia
  Results of the Match [1 0]
   England wins 🏆⚽🎖

England vs Saudi Arabia
  Results of the Match [1 1]
   Match Draw ⚽⚽⚽

 England vs Egypt
  Results of the Match [0 2]
   Egypt wins 🏆⚽🎖

 Russia vs Saudi Arabia
  Results of the Match [0 1]
   Saudi Arabia wins 🏆⚽🎖

 Russia vs Egypt
  Results of the Match [0 1]
   Egypt wins 🏆⚽🎖
```

```
 Saudi Arabia vs Egypt
  Results of the Match [0 5]
   Egypt wins 🏆🏆🏆🏆

Group B Matches
Portugal vs Spain
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Portugal vs Morocco
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 Portugal vs Iran
  Results of the Match [1 2]
   Iran wins 🏆🏆🏆🏆

Spain vs Morocco
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Spain vs Iran
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 Morocco vs Iran
  Results of the Match [0 1]
   Iran wins 🏆🏆🏆🏆

Group C Matches
 France vs Denmark
  Results of the Match [0 2]
   Denmark wins 🏆🏆🏆🏆

 France vs Australia
  Results of the Match [1 2]
   Australia wins 🏆🏆🏆🏆

France vs Peru
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 Denmark vs Australia
  Results of the Match [1 2]
   Australia wins 🏆🏆🏆🏆

 Denmark vs Peru
  Results of the Match [1 0]
   Denmark wins 🏆🏆🏆🏆

Australia vs Peru
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Group D Matches
```

```
Argentina vs Croatia
  Results of the Match [1 1]
   Match Draw ⚽⚽⚽

 Argentina vs Iceland
  Results of the Match [2 1]
   Argentina wins 🏆⚽🎖

Argentina vs Nigeria
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 Croatia vs Iceland
  Results of the Match [0 1]
   Iceland wins 🏆⚽🎖

Croatia vs Nigeria
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Iceland vs Nigeria
  Results of the Match [1 1]
   Match Draw ⚽⚽⚽

Group E Matches
Brazil vs Switzerland
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Brazil vs Costa Rica
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Brazil vs Serbia
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Switzerland vs Costa Rica
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Switzerland vs Serbia
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 Costa Rica vs Serbia
  Results of the Match [1 0]
   Costa Rica wins 🏆⚽🎖

Group F Matches
South Korea vs Mexico
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

South Korea vs Sweden
  Results of the Match [0 0]
```

```
   Match Draw ⚽⚽⚽

 South Korea vs Germany
  Results of the Match [0 1]
   Germany wins 🏆🏆⚽🎖️

Mexico vs Sweden
  Results of the Match [1 1]
   Match Draw ⚽⚽⚽

 Mexico vs Germany
  Results of the Match [1 0]
   Mexico wins 🏆🏆⚽🎖️

Sweden vs Germany
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Group G Matches
Belgium vs England
  Results of the Match [1 1]
   Match Draw ⚽⚽⚽

Belgium vs Panama
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

Belgium vs Tunisia
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽

 England vs Panama
  Results of the Match [1 0]
   England wins 🏆🏆⚽🎖️

 England vs Tunisia
  Results of the Match [1 0]
   England wins 🏆🏆⚽🎖️

 Panama vs Tunisia
  Results of the Match [0 1]
   Tunisia wins 🏆🏆⚽🎖️

Group H Matches
 Senegal vs Japan
  Results of the Match [0 1]
   Japan wins 🏆🏆⚽🎖️

 Senegal vs Poland
  Results of the Match [0 1]
   Poland wins 🏆🏆⚽🎖️

Senegal vs Colombia
  Results of the Match [0 0]
   Match Draw ⚽⚽⚽
```

```
Japan vs Poland
 Results of the Match [1 0]
  Japan wins 🏵🏆⚽🎖

Japan vs Colombia
 Results of the Match [0 1]
  Colombia wins 🏵🏆⚽🎖

Poland vs Colombia
 Results of the Match [0 2]
  Colombia wins 🏵🏆⚽🎖
```

In [58]:

```
##Round of 16 Section_1

qualified_teams_1=[]
standings=list(Group_standings.keys())
i=0
print(f"Round of 16\n")
while i < (len(standings)):
    A_team= sorted(Group_standings[standings[i]][3].items(), key=lambda
x: x[1], reverse=True)
    team_1=A_team[0][0]
    B_team= sorted(Group_standings[standings[i+1]][3].items(),
key=lambda x: x[1], reverse=True)
    team_2=B_team[1][0]

    team_1_num=label_encoder.transform([team_1])[0]
    team_2_num=label_encoder.transform([team_2])[0]
    team_lst=[team_1,team_2]

    Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
    res=model.predict(Input_vector)
    win,_=select_winning_team(res)

    try:
            print(f"{team_1} vs {team_2} \n  {team_lst[win]} wins 🏆
🏆")
            print(f"    {team_lst[win]} into the Quater-Finals ⏭⏭
\n")
            qualified_teams_1.append(team_lst[win])
    except IndexError:
            print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
            winning_team=random.choice(team_lst)
            print(f"   {winning_team} wins at Penalty Shoot-Out 🥅🎯")
```

```
            print(f"    {winning_team} into the Quater-Finals ⏭⏭ \n")
            qualified_teams_1.append(winning_team)
    i=i+2

##Round of 16 Section_2
qualified_teams_2=[]
standings=list(Group_standings.keys())
i=0
while i < (len(standings)):
    A_team= sorted(Group_standings[standings[i]][3].items(), key=lambda
x: x[1], reverse=True)
    team_1=A_team[1][0]
    B_team= sorted(Group_standings[standings[i+1]][3].items(),
key=lambda x: x[1], reverse=True)
    team_2=B_team[0][0]

    team_1_num=label_encoder.transform([team_1])[0]
    team_2_num=label_encoder.transform([team_2])[0]
    team_lst=[team_1,team_2]

    Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
    res=model.predict(Input_vector)
    win,_=select_winning_team(res)

    try:
            print(f"{team_1} vs {team_2} \n  {team_lst[win]} wins 🏆
🏆")
            print(f"    {team_lst[win]} into the Quater-Finals ⏭⏭
\n")
            qualified_teams_2.append(team_lst[win])

    except IndexError:
            print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
            winning_team=random.choice(team_lst)
            print(f"   {winning_team} wins at Penalty Shoot-Out 🥅🎯")
            print(f"    {winning_team} into the Quater-Finals ⏭⏭ \n")
            qualified_teams_2.append(winning_team)
    i=i+2
```

```
Round of 16

Egypt vs Spain
  Egypt wins 🏆🏆
```

```
    Egypt into the Quater-Finals ⏩⏩

Australia vs Iceland
  Iceland wins 🏆🏆
    Iceland into the Quater-Finals ⏩⏩

Costa Rica vs Sweden
  Costa Rica wins 🏆🏆
    Costa Rica into the Quater-Finals ⏩⏩

England vs Japan
  England wins 🏆🏆
    England into the Quater-Finals ⏩⏩

England vs Iran
  Match Draw ⚽⚽
   Iran wins at Penalty Shoot-Out 🥅🎯
    Iran into the Quater-Finals ⏩⏩

Denmark vs Argentina
  Match Draw ⚽⚽
   Argentina wins at Penalty Shoot-Out 🥅🎯
    Argentina into the Quater-Finals ⏩⏩

Brazil vs Mexico
  Match Draw ⚽⚽
   Brazil wins at Penalty Shoot-Out 🥅🎯
    Brazil into the Quater-Finals ⏩⏩

Belgium vs Colombia
  Belgium wins 🏆🏆
    Belgium into the Quater-Finals ⏩⏩
```

In [72]:

```
print(f"Teams selected to the Quater Finals -
{qualified_teams_1+qualified_teams_2}")
```

```
Teams selected to the Quater Finals - ['Egypt', 'Iceland', 'Costa Rica', 'England',
'Iran', 'Argentina', 'Brazil', 'Belgium']
```

In [60]:

```
Semifinal_teams=[]
i=0
print(f"Quater Final Matches\n")
while i < (len(qualified_teams_1))-1:
    team_1= qualified_teams_1[i]
    team_2= qualified_teams_1[i+1]

    team_1_num=label_encoder.transform([team_1])[0]
    team_2_num=label_encoder.transform([team_2])[0]
    team_lst=[team_1,team_2]
```

```
    Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
    res=model.predict(Input_vector)
    win,_=select_winning_team(res)

    try:
            print(f"{team_1} vs {team_2} \n  {team_lst[win]} wins 🏆
🏆")
            print(f"    {team_lst[win]} into the Semi-Finals ⏭⏭ \n")
            Semifinal_teams.append(team_lst[win])

    except IndexError:
            print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
            winning_team=random.choice(team_lst)
            print(f"   {winning_team} wins at Penalty Shoot-Out 🥅🎯")
            print(f"    {winning_team} into the Semi-Finals ⏭⏭ \n")
            Semifinal_teams.append(winning_team)
    i=i+2

i=0
while i < (len(qualified_teams_2))-1:
    team_1= qualified_teams_2[i]
    team_2= qualified_teams_2[i+1]
    team_1_num=label_encoder.transform([team_1])[0]
    team_2_num=label_encoder.transform([team_2])[0]
    team_lst=[team_1,team_2]

    Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
    res=model.predict(Input_vector)
    win,_=select_winning_team(res)

    try:
            print(f"{team_1} vs {team_2} \n  {team_lst[win]} wins 🏆
🏆")
            print(f"    {team_lst[win]} into the Semi-Finals ⏭⏭ \n")
            Semifinal_teams.append(team_lst[win])

    except IndexError:
            print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
            winning_team=random.choice(team_lst)
            print(f"   {winning_team} wins at Penalty Shoot-Out 🥅🎯")
            print(f"    {winning_team} into the Semi-Finals ⏭⏭ \n")
```

```
            Semifinal_teams.append(winning_team)
    i=i+2
```

```
Quater Final Matches

Egypt vs Iceland
  Match Draw ⚽⚽
   Egypt wins at Penalty Shoot-Out 🥅🎯
    Egypt into the Semi-Finals ⏭⏭

Costa Rica vs England
  England wins 🏆🏆
    England into the Semi-Finals ⏭⏭

Iran vs Argentina
  Match Draw ⚽⚽
   Argentina wins at Penalty Shoot-Out 🥅🎯
    Argentina into the Semi-Finals ⏭⏭

Brazil vs Belgium
  Match Draw ⚽⚽
   Brazil wins at Penalty Shoot-Out 🥅🎯
    Brazil into the Semi-Finals ⏭⏭
```

In [61]:

```
print(f"Teams selected to the Semi-Finals - {Semifinal_teams}")
```

```
Teams selected to the Semi-Finals - ['Egypt', 'England', 'Argentina', 'Brazil']
```

In [62]:

```
#Semi Finals

final_teams=[]
third_place_match_teams=[]
i=0
print(f"Semi Final Matches\n")
while i < (len(Semifinal_teams))-1:
    team_1= Semifinal_teams[i]
    team_2= Semifinal_teams[i+1]

    team_1_num=label_encoder.transform([team_1])[0]
    team_2_num=label_encoder.transform([team_2])[0]
    team_lst=[team_1,team_2]

    Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
    res=model.predict(Input_vector)
    win,_=select_winning_team(res)

    try:
```

```
            print(f"{team_1} vs {team_2} \n  {team_lst[win]} wins 🏆🏆")
            print(f"    {team_lst[win]} into the FiIFA-Finals ⏩⏩ \n")
            final_teams.append(team_lst[win])
            third_place_match_teams.append(team_lst[(win+1)%2])


    except IndexError:
            print(f"{team_1} vs {team_2} \n  Match Draw 🎱🎱")
            winning_team=random.choice(team_lst)
            print(f"   {winning_team} wins at Penaly Shoot-Out 🥅🎯")
            print(f"    {winning_team} into the FIFA-Finals ⏩⏩ \n")
            final_teams.append(winning_team)
            team_lst.remove(winning_team)
            third_place_match_teams.append(team_lst[0])
    i=i+2
```

```
Semi Final Matches

Egypt vs England
  England wins 🏆🏆
    England into the FiIFA-Finals ⏩⏩

Argentina vs Brazil
  Argentina wins 🏆🏆
    Argentina into the FiIFA-Finals ⏩⏩
```

In [63]:

```
print(f"FIFA FINAl\n")
team_1= final_teams[1]
team_2= final_teams[0]

team_1_num=label_encoder.transform([team_1])[0]
team_2_num=label_encoder.transform([team_2])[0]
team_lst=[team_1,team_2]

Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
res=model.predict(Input_vector)
win,_=select_winning_team(res)

try:
    print(f"{team_1} vs {team_2} \n  {team_lst[win]} are the Winners 🎉🏆🎉\n\n")
    winner=team_lst[win]
    place_2=team_lst[(win+1)%2]
```

```
except IndexError:
    print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
    winning_team=random.choice(team_lst)
    print(f"   {winning_team} wins at Penaly Shoot-Out 🥅🎯")
    print(f"    {winning_team} are the Winners 🎉🏆🎉\n\n")
    winner=winning_team

    team_lst.remove(winning_team)
    place_2=team_lst[0]

print(f"Third Place match\n")
team_1= third_place_match_teams[1]
team_2= third_place_match_teams[0]

team_1_num=label_encoder.transform([team_1])[0]
team_2_num=label_encoder.transform([team_2])[0]
team_lst=[team_1,team_2]

Input_vector=np.array([[year,host_num,team_1_num,team_2_num]])
res=model.predict(Input_vector)
win,_=select_winning_team(res)

try:
    print(f"{team_1} vs {team_2} \n  {team_lst[win]} Wins the 3rd Place
🎉🏆🎉\n")
    place_3=team_lst[win]

except IndexError:
    print(f"{team_1} vs {team_2} \n  Match Draw ⚽⚽")
    winning_team=random.choice(team_lst)
    print(f"   {winning_team} wins at Penaly Shoot-Out 🥅🎯")
    print(f"    {winning_team} Wins the 3rd Place 🎉🏆🎉\n")
    place_3=winning_team



print(f"\n\nWinner is {winner} 🥇🥇🥇")
print(f"Runner-up is {place_2} 🥈🥈🥈")
print(f"3rd Place is {place_3} 🥉🥉🥉")
```

```
FIFA FINAl
```

```
Argentina vs England
  England are the Winners 🎉🏆🎉


Third Place match

Brazil vs Egypt
  Match Draw ⚽⚽
   Brazil wins at Penaly Shoot-Out 🥅🎯
    Brazil Wins the 3rd Place 🎉🏆🎉



Winner is England 🥇🥇🥇
Runner-up is Argentina 🥈🥈🥈
3rd Place is Brazil 🥉🥉🥉
```

In [ ]:

```
##ML mini project by yashashwini S Awate -TY-A-04 batch 2021
```